LAPORAN PRAKTIKUM MICROPROCESSOR & INTERFACE

PROGRAM KONVERSI DATA HEKSADESIMAL KE OKTAL DAN OKTAL KE HEKSADESIMAL



PRAKTIKAN

Nama : Elsadanika Maura Sitorus

NIM : 211344013

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2022/2023

KETERANGAN

1. Kelompok : -
2. Judul Laporan : Konversi Data Heksadesimal ke Oktal dan Oktal ke Heksadesimal
3. Tanggal Praktikum : 28 Februari 2023
4. Tanggal Pengumpulan : 04 Maret 2023
5. Nama Praktikan : Elsadanika Maura Sitorus
6. Nama Partner : -
7. Nama Dosen Pembimbing : Ferry Satria, BSEE., M. T.
   Rahmawati Hasanah, S. ST., M. T.

DAFTAR ISI

KETERANGAN i
DAFTAR ISI ii

A. Tujuan Praktikum 1
B. Dasar Teori 1
C. Alat dan Komponen yang Digunakan 3
D. Langkah Kerja 3
E. Data Pengamatan 4
F. Analisis Data 20
G. Kesimpulan 26
H. Daftar Pustaka 26

A. Tujuan Praktikum

1. Memahami bahasa Assembly
2. Memahami cara penggunaan simulator EMU 8086
3. Memahami pemanfaatan EMU 8086 simulator untuk konversi data bilangan hexadecimal dan oktal menggunakan bahasa pemrograman Assembly.

B. Dasar Teori

1. Perintah Looping, Memory, dan Data Area.

Arahan assembler define digunakan sebagai alokasi ruang penyimpanan dan untuk menginisialisasi satu atau lebih byte.

a) Mengalokasikan Ruang Penyimpanan untuk Data yang Diinisialisasi

Sintaks untuk pernyataan alokasi penyimpanan untuk data yang diinisialisasi adalah sebagai berikut:

```
[nama-variabel] define-directive initial-value [,initial-value]...
```

Nama variabel adalah pengidentifikasi untuk setiap ruang penyimpanan. Assembler mengaitkan nilai offset untuk setiap nama variabel yang ditentukan dalam segmen data. Ada lima bentuk dasar dari direktif define yang ditampilkan pada tabel berikut:

Perintah	Keterangan	Storage Space
DB	Define Byte	1 byte
DW	Define Word	2 bytes
DD	Define Doubleword	4 bytes
DQ	Define Quadword	8 bytes
DT	Define Ten Bytes	10 bytes

b) Perintah Looping

Instruksi untuk transfer perintah selama eksekusi disesuaikan dengan kondisinya, berikut ini adalah instruksi untuk proses looping dalam Bahasa assembly pada 8086:

Instructions	Conditions
JA/JNBE	Jump if above/not below/equal instruction satisfies
JAE/JNB	Jump if above/not below
JBE/JNA	Jump if below/equal/ not above

JC	Jump if carry flag CF = 1
JE/JZ	Jump if equal/zero flag ZF = 1
JG/JNLE	Jump if greater/not less than/equal
JGE/JNL	Jump if greater than/equal/not less
JL/JNGE	Jump if less than/not greater than/equal
JLE/JNG	Jump if less than/equal/if not greater
JNC	Jump if no carry flag (CF = 0)
JNE/JNZ	Jump if not equal/zero flag ZF = 0
JNO	Jump if no overflow flag OF = 0
JNP/JPO	Jump if not parity/parity odd PF = 0
JNS	Jump if not sign SF = 0
JO	Jump if overflow flag OF = 1
JP/JPE	Jump if parity/parity even PF = 1
JS	Jump if sign flag SF = 1

2. Perintah Konversi Data Heksadesimal ke Oktal

Konversi data dari heksadesimal ke oktal menggunakan metode penyisipan 0 untuk setiap 3 bit bilangan pada data heksadesimal yang telah berbentuk biner. Contoh konsep pengerjaan dari konversi ini adalah sebagai berikut:

$3F_{(16)}$ = 00111111

Bilangan biner dari 3F disusun per 3 bit, seperti berikut:

00 111 111 = 077

Sehingga $3F_{(16)} = 77_{(8)}$

Agar dapat dikonversi ke oktal, maka perlu disisipkan 0 pada tiap 3 bit data, sehingga bentuk binernya menjadi seperti berikut:

000 01110111 → nilai tetap 77, tidak berubah.

Secara program computer, proses yang perlu dilakukan untuk menyisipkan 0 tiap 3 digit adalah melalui perintah AND dengan 7 untuk mengambil 3 digit angka terakhir, lalu digeser ke kanan sebanyak 1 melalui perintah SHR. Jika sudah maka AND kembali dengan F8 supaya 3 digit terakhir bernilai 0.

Perumusan untuk 4 digit dalam 1 data heksadesimal yang akan diubah ke oktal adalah sebagai berikut:

X = ((((((d1*16)+d2)*16)+d3)*16)+d4)

3. Perintah Konversi Data Oktal ke Heksadesimal

Konversi data oktal ke heksadesimal memiliki konsep yang serupa dengan konversi data decimal ke heksadesimal. Perbedaannya hanya terdapat pada jumlah

penggandaan. Data oktal memiliki basis 8 sehingga hanya perlu penggandaan sebanyak 3 kali untuk mendapat faktor pengali 8. Konsep penggandaan untuk mendapat pengali 8 adalah sebagai berikut:

(A*8) = A * (2+2+2)

2 = penggandaan 1 kali

Sehingga data perlu digandakan sebanyak 3 kali untuk mendapat hasil yang sama dengan pengali 8.

Selain itu, terdapat cara lain untuk mengkonversi data oktal ke heksadesimal. Dengan menghapus bit 0 dari bilangan data oktal sehingga dihasilkan bilangan data heksadesimal yang ditunjukkan pada contoh berikut ini:

77 = 0111 0111 Oktal maka akan didapatkan bilangan heksadesimal dengan cara menghapus angka 0 sehingga menjadi 0111111 oktal telah diubah ke heksadesimal dan didapat data heksadesimal 3F.

C. Alat dan Komponen yang Digunakan

- Laptop/Personal Computer
- Simulator EMU 8086

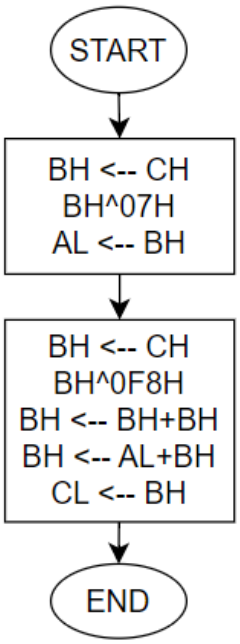
D. Langkah Kerja

1. Buat diagram alir dan program untuk mengubah data HEKSADESIMAL 2 digit yang ada pada register CH menjadi OKTAL. Simpan hasil konversi pada register DL.
2. Buat diagram alir dan program untuk mengubah data HEKSADESIMAL 4 digit yang ada pada register CX menjadi OKTAL. Simpan hasil konversi pada register CL:DX.
3. Buat diagram alir dan program untuk mengubah 16 data HEKSADESIMAL 2 digit yang ada pada BM1 menjadi OKTAL. Simpan hasil konversi pada BM2.
4. Buat diagram alir dan program untuk mengubah 8 data HEKSADESIMAL 4 digit yang ada pada BM1 menjadi OKTAL. Simpan hasil konversi pada BM2.
5. Buat diagram alir dan program untuk mengubah 6 data HEKSADESIMAL 6 digit yang ada pada BM1 menjadi OKTAL. Simpan hasil konversi pada BM2.
6. Buat diagram alir dan program untuk mengubah 6 data HEKSADESIMAL 8 digit yang ada pada BM1 menjadi OKTAL. Simpan hasil konversi pada BM2.
7. Buat diagram alir dan program untuk mengubah 6 data HEKSADESIMAL 10 digit yang ada pada BM1 menjadi OKTAL. Simpan hasil konversi pada BM2.
8. Buat diagram alir dan program untuk mengubah 16 data OKTAL 2 digit yang ada pada BM1 menjadi HEKSADESIMAL. Simpan hasil konversi pada BM2.
9. Buat diagram alir dan program untuk mengubah 10 data OKTAL 4 digit yang ada pada BM1 menjadi HEKSADESIMAL. Simpan hasil konversi pada BM2.
10. Buat diagram alir dan program untuk mengubah 8 data OKTAL 6 digit yang ada pada BM1 menjadi HEKSADESIMAL. Simpan hasil konversi pada BM2.

11. Buat diagram alir dan program untuk mengubah 6 data OKTAL 10 digit yang ada pada BM1 menjadi HEKSADESIMAL. Simpan hasil konversi pada BM2.

E. Data Pengamatan

1. Program 1



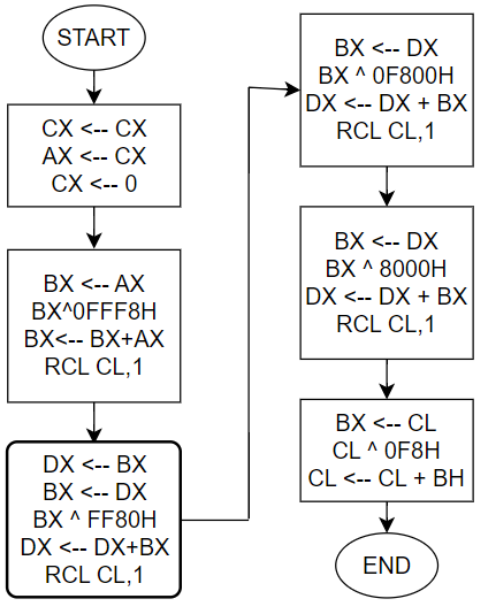

```
TITLE HEX KE OCT 2DIGIT ;IN CH OUT DL
.MODEL SMALL
.STACK 100H
.DATA

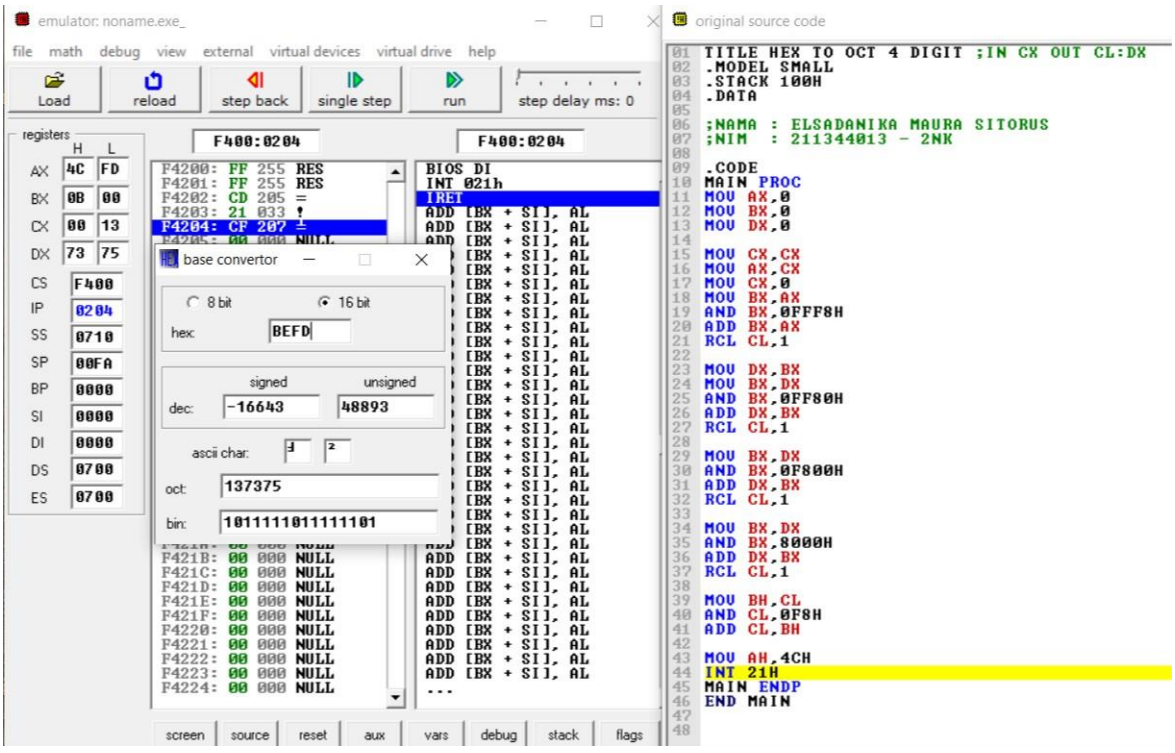
;NAMA : ELSADANIKA MAURA
;NIM  : 211344013 - 2NK

.CODE
MAIN PROC
MOV BH,CH
AND BH,07H
MOV AL,BH
MOV BH,CH
AND BH,0F8H
ADD BH,BH
ADD BH,AL
MOV CL,BH

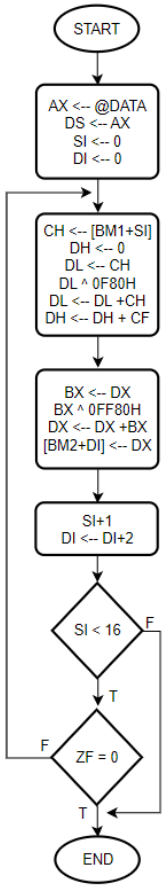
MOV AH,4CH
INT 21H
MAIN ENDP
END MAIN
```

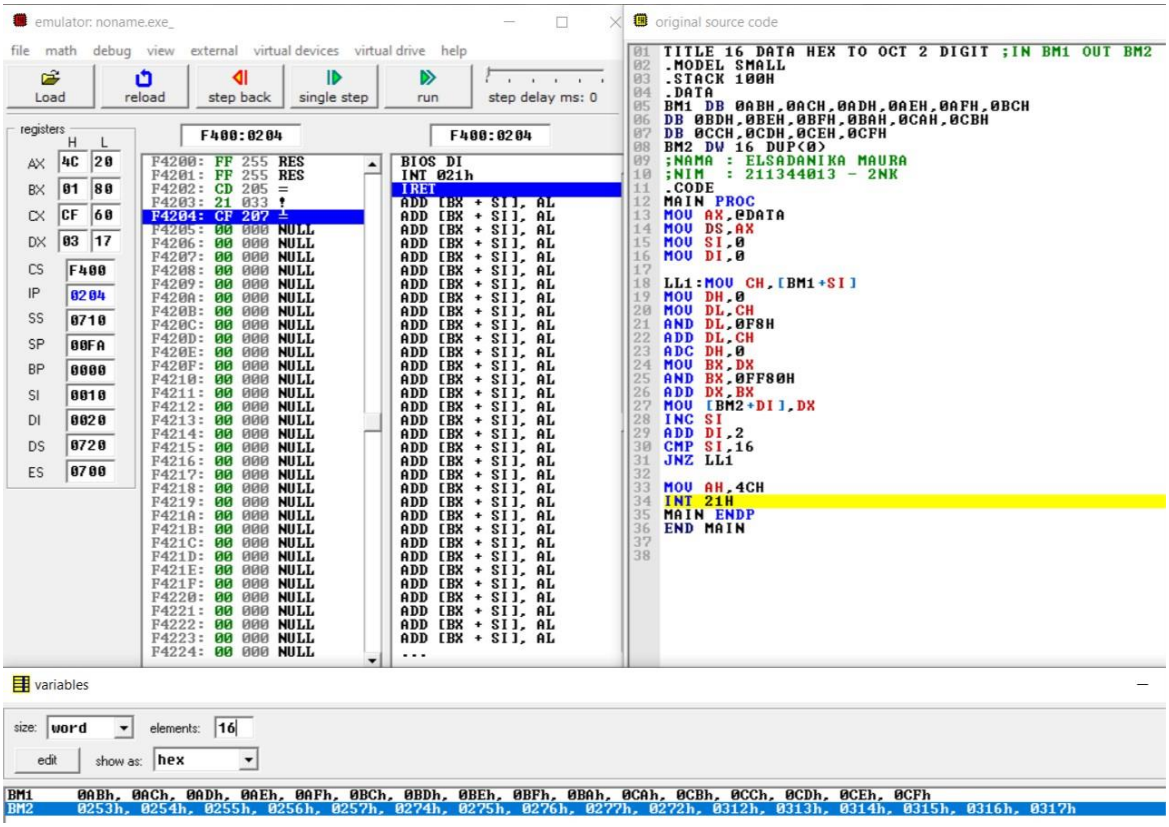
2. Program 2


START
CX <-- CX
AX <-- CX
CX <-- 0
BX <-- AX
BX^0FFF8H
BX<-- BX+AX
RCL CL,1
DX <-- BX
BX <-- DX
BX ^ FF80H
DX <-- DX+BX
RCL CL,1
BX <-- DX
BX ^ 0F800H
DX <-- DX + BX
RCL CL,1
BX <-- DX
BX ^ 8000H
DX <-- DX + BX
RCL CL,1
BX <-- CL
CL ^ 0F8H
CL <-- CL + BH
END

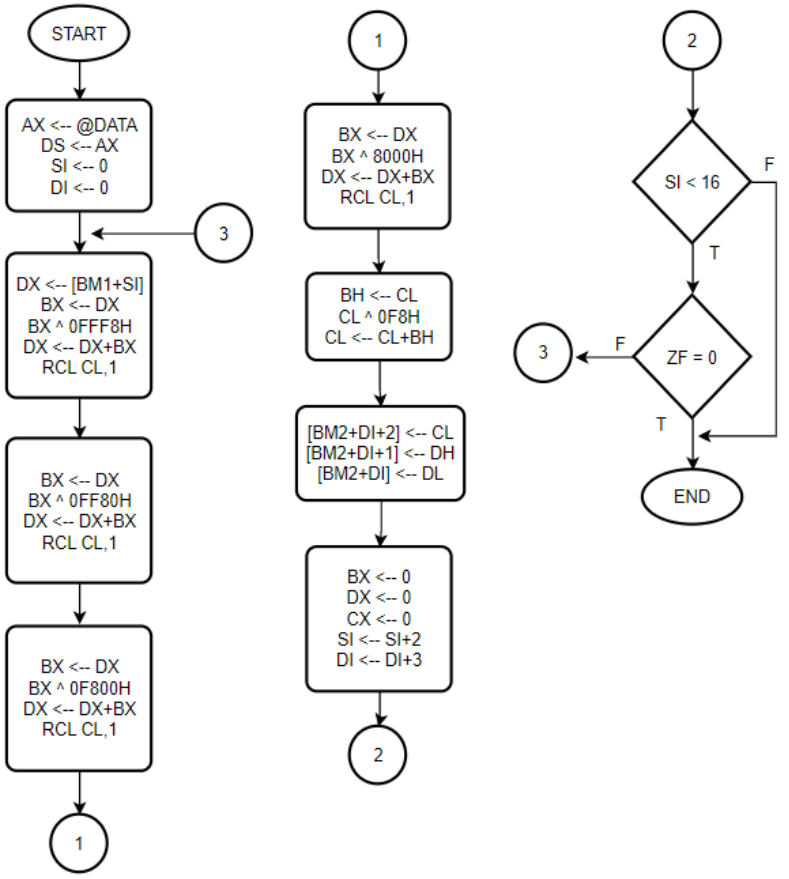

emulator: noname.exe_
file math debug view external virtual devices virtual drive help
Load reload step back single step run step delay ms: 0
registers
AX 4C FD
BX 0B 00
CX 00 13
DX 73 75
CS F400
IP 0204
SS 0710
SP 00FA
BP 0000
SI 0000
DI 0000
DS 0700
ES 0700
F400:0204
F4200: FF 255 RES
F4201: FF 255 RES
F4202: CD 205 =
F4203: 21 033 !
F4204: CF 207 ≟
base convertor
8 bit
16 bit
hex: BEFD
signed unsigned
dec: -16643 48893
ascii char:
oct: 137375
bin: 1011111011111101
BIOS DI
INT 021h
IRET
ADD [BX + SI], AL
screen source reset aux vars debug stack flags
original source code
TITLE HEX TO OCT 4 DIGIT ;IN CX OUT CL:DX
.MODEL SMALL
.STACK 100H
.DATA
;NAMA : ELSADANIKA MAURA SITORUS
;NIM : 211344013 - 2NK
.CODE
MAIN PROC
MOV AX,0
MOV BX,0
MOV DX,0
MOV CX,CX
MOV AX,CX
MOV CX,0
MOV BX,AX
AND BX,0FFF8H
ADD BX,AX
RCL CL,1
MOV DX,BX
MOV BX,DX
AND BX,0FF80H
ADD DX,BX
RCL CL,1
MOV BX,DX
AND BX,0F800H
ADD DX,BX
RCL CL,1
MOV BX,DX
AND BX,8000H
ADD DX,BX
RCL CL,1
MOV BH,CL
AND CL,0F8H
ADD CL,BH
MOV AH,4CH
INT 21H
MAIN ENDP
END MAIN

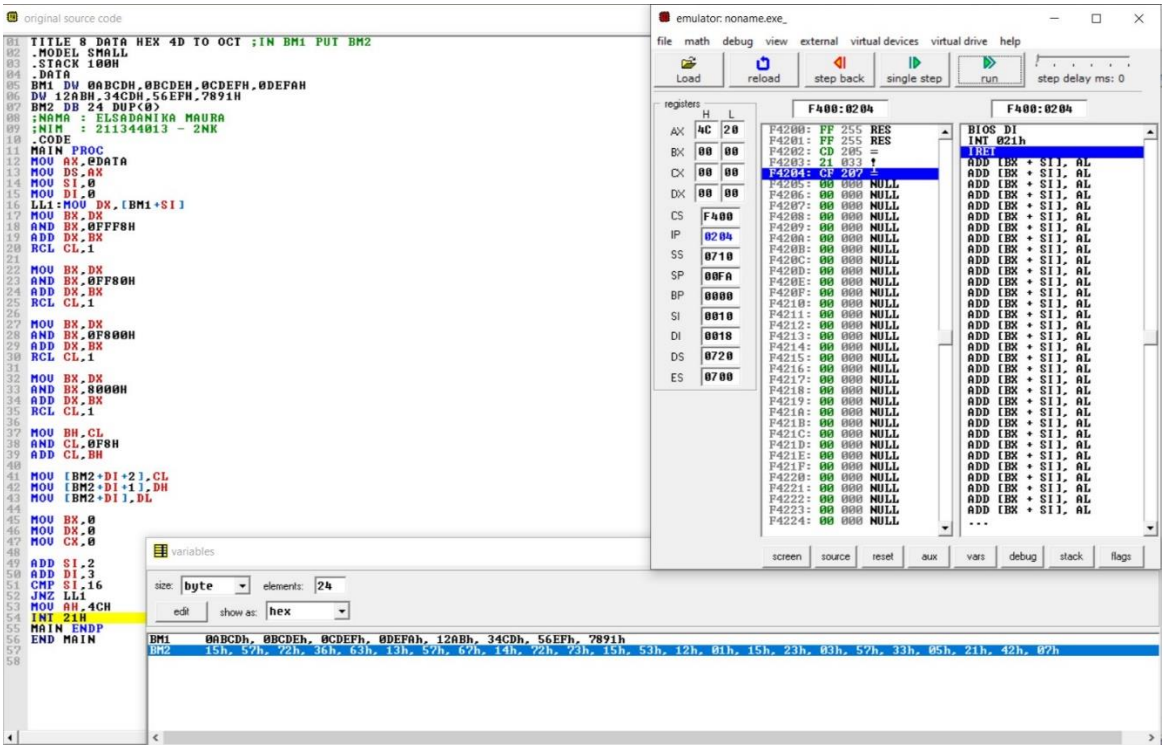
3. Program 3


START
AX <-- @DATA
DS <-- AX
SI <-- 0
DI <-- 0
CH <-- [BM1+SI]
DH <-- 0
DL <-- CH
DL ^ 0F80H
DL <-- DL +CH
DH <-- DH + CF
BX <-- DX
BX ^ 0FF80H
DX <-- DX +BX
[BM2+DI] <-- DX
SI+1
DI <-- DI+2
SI < 16
F
T
ZF = 0
F
T
END

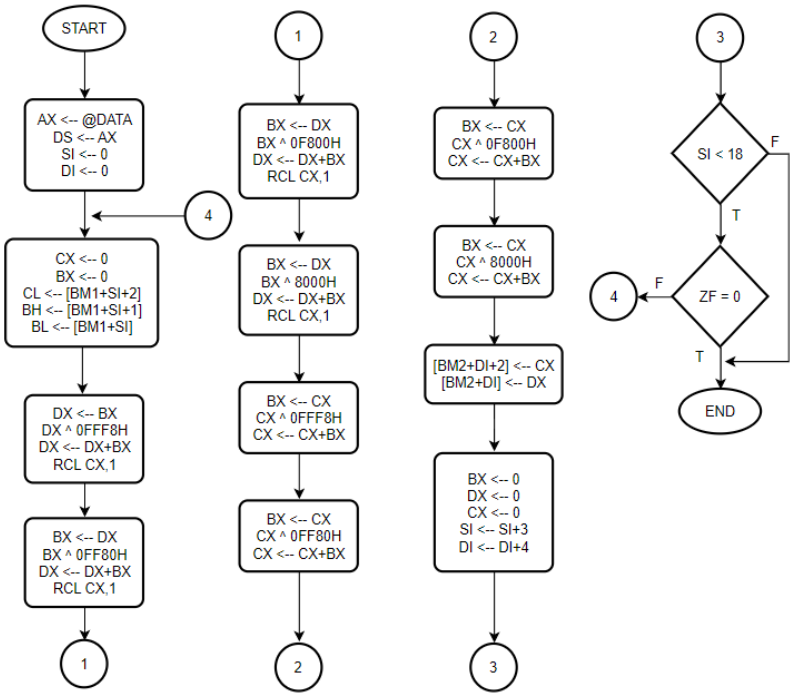

emulator: noname.exe_
file math debug view external virtual devices virtual drive help
Load reload step back single step run step delay ms: 0
registers
AX 4C 20
BX 01 80
CX CF 60
DX 03 17
CS F400
IP 0204
SS 0710
SP 00FA
BP 0000
SI 0010
DI 0020
DS 0720
ES 0700
F400:0204
F400:0204
BIOS DI
INT 021h
IRET
original source code
TITLE 16 DATA HEX TO OCT 2 DIGIT ;IN BM1 OUT BM2
.MODEL SMALL
.STACK 100H
.DATA
BM1 DB 0ABH,0ACH,0ADH,0AEH,0AFH,0BCH
DB 0BDH,0BEH,0BFH,0BAH,0CAH,0CBH
DB 0CCH,0CDH,0CEH,0CFH
BM2 DW 16 DUP(0)
;NAMA : ELSADANIKA MAURA
;NIM : 211344013 - 2NK
.CODE
MAIN PROC
MOV AX,@DATA
MOV DS,AX
MOV SI,0
MOV DI,0
LL1:MOV CH,[BM1+SI]
MOV DH,0
MOV DL,CH
AND DL,0F8H
ADD DL,CH
ADC DH,0
MOV BX,DX
AND BX,0FF80H
ADD DX,BX
MOV [BM2+DI],DX
INC SI
ADD DI,2
CMP SI,16
JNZ LL1
MOV AH,4CH
INT 21H
MAIN ENDP
END MAIN
variables
size: word elements: 16
edit show as: hex
BM1 0ABh, 0ACh, 0ADh, 0AEh, 0AFh, 0BCh, 0BDh, 0BEh, 0BFh, 0BAh, 0CAh, 0CBh, 0CCh, 0CDh, 0CEh, 0CFh
BM2 0253h, 0254h, 0255h, 0256h, 0257h, 0274h, 0275h, 0276h, 0277h, 0272h, 0312h, 0313h, 0314h, 0315h, 0316h, 0317h

4. Program 4


START
AX <-- @DATA
DS <-- AX
SI <-- 0
DI <-- 0
3
DX <-- [BM1+SI]
BX <-- DX
BX ^ 0FFF8H
DX <-- DX+BX
RCL CL,1
BX <-- DX
BX ^ 0FF80H
DX <-- DX+BX
RCL CL,1
BX <-- DX
BX ^ 0F800H
DX <-- DX+BX
RCL CL,1
1
1
BX <-- DX
BX ^ 8000H
DX <-- DX+BX
RCL CL,1
BH <-- CL
CL ^ 0F8H
CL <-- CL+BH
[BM2+DI+2] <-- CL
[BM2+DI+1] <-- DH
[BM2+DI] <-- DL
BX <-- 0
DX <-- 0
CX <-- 0
SI <-- SI+2
DI <-- DI+3
2
2
SI < 16
F
T
3
F
ZF = 0
T
END


original source code
TITLE 8 DATA HEX 4D TO OCT ;IN BM1 PUT BM2
.MODEL SMALL
.STACK 100H
.DATA
BM1 DW 0ABCDH,0BCDEH,0CDEFH,0DEFAH
DW 12ABH,34CDH,56EFH,7891H
BM2 DB 24 DUP(0)
;NAMA : ELSADANIKA MAURA
;NIM : 211344013 - 2NK
.CODE
MAIN PROC
MOV AX,@DATA
MOV DS,AX
MOV SI,0
MOV DI,0
LL1:MOV DX,[BM1+SI]
MOV BX,DX
AND BX,0FFF8H
ADD DX,BX
RCL CL,1
MOV BX,DX
AND BX,0FF80H
ADD DX,BX
RCL CL,1
MOV BX,DX
AND BX,0F800H
ADD DX,BX
RCL CL,1
MOV BX,DX
AND BX,8000H
ADD DX,BX
RCL CL,1
MOV BH,CL
AND CL,0F8H
ADD CL,BH
MOV [BM2+DI+2],CL
MOV [BM2+DI+1],DH
MOV [BM2+DI],DL
MOV BX,0
MOV DX,0
MOV CX,0
ADD SI,2
ADD DI,3
CMP SI,16
JNZ LL1
MOV AH,4CH
INT 21H
MAIN ENDP
END MAIN
emulator: noname.exe_
file math debug view external virtual devices virtual drive help
Load reload step back single step run step delay ms: 0
registers
AX 4C 20
BX 00 00
CX 00 00
DX 00 00
CS F400
IP 0204
SS 0710
SP 00FA
BP 0000
SI 0010
DI 0018
DS 0720
ES 0700
F400:0204
F400:0204
BIOS DI
INT 021h
IRET
screen source reset aux vars debug stack flags
variables
size: byte elements: 24
edit show as: hex
BM1 0ABCDh, 0BCDEh, 0CDEFh, 0DEFAh, 12ABh, 34CDh, 56EFh, 7891h
BM2 15h, 57h, 72h, 36h, 63h, 13h, 57h, 67h, 14h, 72h, 73h, 13h, 53h, 12h, 01h, 15h, 23h, 03h, 57h, 33h, 05h, 21h, 42h, 07h

5. Program 5



original source code

```
01 TITLE 6 DATA HEX 6D TO OCT ;IN BM1 PUT BM2
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 BM1 DB 0ABH,0CDH,0BCH, 0DEH,0CDH,0EFH, 0DEH,0FAH,11H
06 DB 12H,0ABH,34H, 0CDH,56H,0EFH, 78H,91H,0AAH
07 BM2 DW 12 DUP(0)
08 ;NAMA : ELSADANIKA MAURA
09 ;NIM  : 211344013 - 2NK
10 .CODE
11 MAIN PROC
12 MOV AX,@DATA
13 MOV DS,AX
14 MOV SI,0
15 MOV DI,0
16 LL1:MOV CX,0
17 MOV BX,0
18 MOV CL,[BM1+SI+2]
19 MOV BH,[BM1+SI+1]
20 MOV BL,[BM1+SI]
21 MOV DX,BX
22 AND DX,0FFF8H
23 ADD DX,BX
24 RCL CX,1
25
26 MOV BX,DX
27 AND BX,0FF80H
28 ADD DX,BX
29 RCL CX,1
30
31 MOV BX,DX
32 AND BX,0F800H
33 ADD DX,BX
34 RCL CX,1
35
36 MOV BX,DX
37 AND BX,8000H
38 ADD DX,BX
39 RCL CX,1
40
41 MOV BX,CX
42 AND CX,0FFF8H
43 ADD CX,BX
44
45 MOV BX,CX
46 AND BX,0FF80H
47 ADD CX,BX
48
49 MOV BX,CX
50 AND CX,0F800H
51 ADD CX,BX
52
53 MOV BX,CX
54 AND CX,8000H
55 ADD CX,BX
56
57 MOV [BM2+DI+2],CX
58 MOV [BM2+DI],DX
59
60 MOV BX,0
61 MOV DX,0
62 MOV CX,0
```

```
64 ADD SI,3
65 ADD DI,4
66 CMP SI,18
67 JNZ LL1
68 MOV AH,4CH
69 INT 21H
70 MAIN ENDP
71 END MAIN
72
73
```

variables

size: word elements: 12

edit show as: hex

BM1	0ABh, 0CDh, 0BCh, 0DEh, 0CDh, 0EFh, 0DEh, 0FAh, 11h, 12h, 0ABh, 34h, 0CDh, 56h, 0EFh, 78h, 91h, 0AAh
BM2	6653h, 5714h, 6736h, 7374h, 5336h, 0437h, 5422h, 1512h, 3315h, 7365h, 0570h, 5251h

Enter hex number

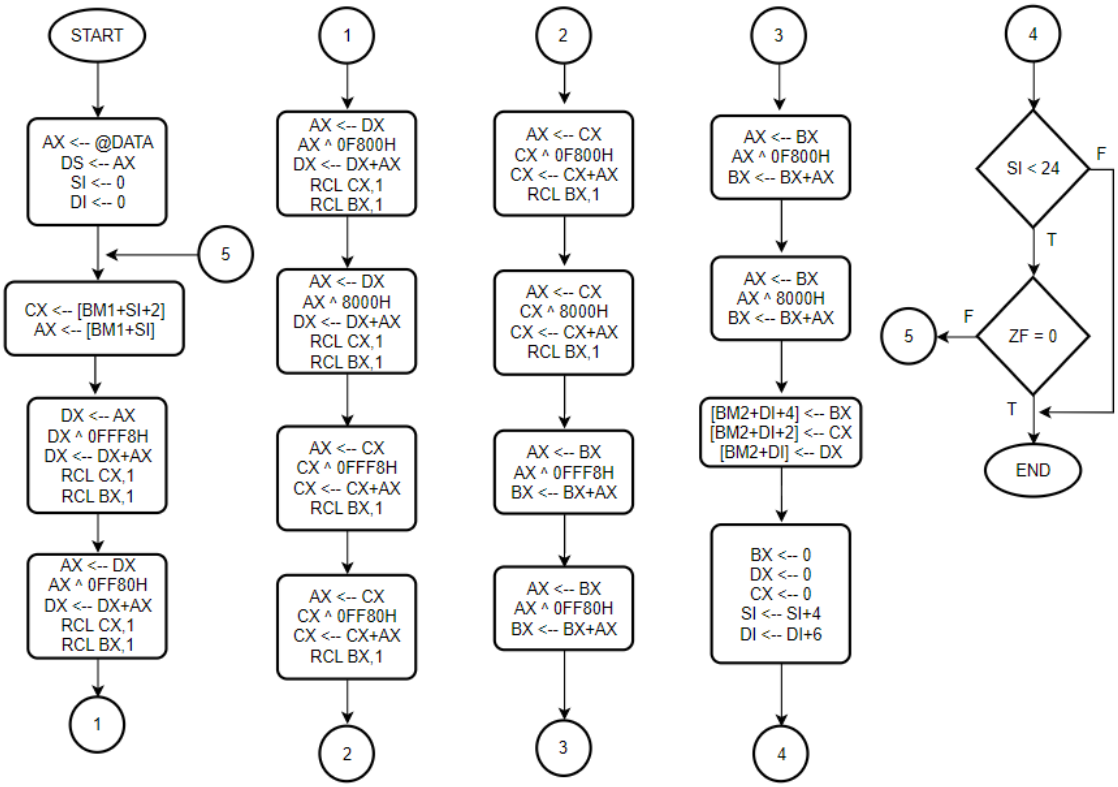
EFCDDE 16

= Convert × Reset ⇅ Swap

Octal number

73746736 8

6. Program 6




```
01 TITLE 6 DATA HEX 8D TO OCT ;IN BM1 PUT BM2
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 BM1 DW 0ABCDH,0BCDEH, 0CDEFH,0DEFAH, 1122H,3344H
06     DW 12ABH,34CDH, 56EFH,7891H, 0AABBH,0CCDDH
07 BM2 DW 18 DUP(0)
08 ;NAMA  : ELSADANIKA MAURA
09 ;NIM   : 211344013 - 2NK
10 .CODE
11 MAIN PROC
12      MOV AX,@DATA
13      MOV DS,AX
14      MOV SI,0
15      MOV DI,0
16 LL1:MOV CX,[BM1+SI+2]
17      MOV AX,[BM1+SI]
18      MOV DX,AX
19      AND DX,0FFF8H
20      ADD DX,AX
21      RCL CX,1
22      RCL BX,1
23
24      MOV AX,DX
25      AND AX,0FF80H
26      ADD DX,AX
27      RCL CX,1
28      RCL BX,1
29
30      MOV AX,DX
31      AND AX,0F800H
32      ADD DX,AX
33      RCL CX,1
34      RCL BX,1
35
36      MOV AX,DX
37      AND AX,8000H
38      ADD DX,AX
39      RCL CX,1
40      RCL BX,1
41
42      MOV AX,CX
43      AND CX,0FFF8H
44      ADD CX,AX
45      RCL BX,1
46
47      MOV AX,CX
48      AND AX,0FF80H
49      ADD CX,AX
50      RCL BX,1
51
52      MOV AX,CX
53      AND CX,0F800H
54      ADD CX,AX
55      RCL BX,1
56
57      MOV AX,CX
58      AND CX,8000H
```

```
    ADD CX,AX
    RCL BX,1

    MOV AX,BX
    AND AX,0FFF8H
    ADD BX,AX

    MOV AX,BX
    AND AX,0FF80H
    ADD BX,AX

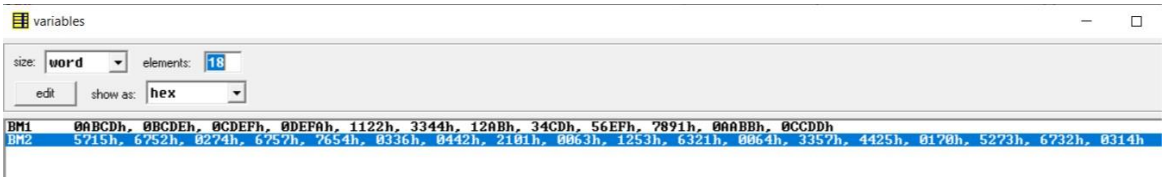
    MOV AX,BX
    AND AX,0F800H
    ADD BX,AX

    MOV AX,BX
    AND AX,8000H
    ADD BX,AX

    MOV [BM2+DI+4],BX
    MOV [BM2+DI+2],CX
    MOV [BM2+DI],DX

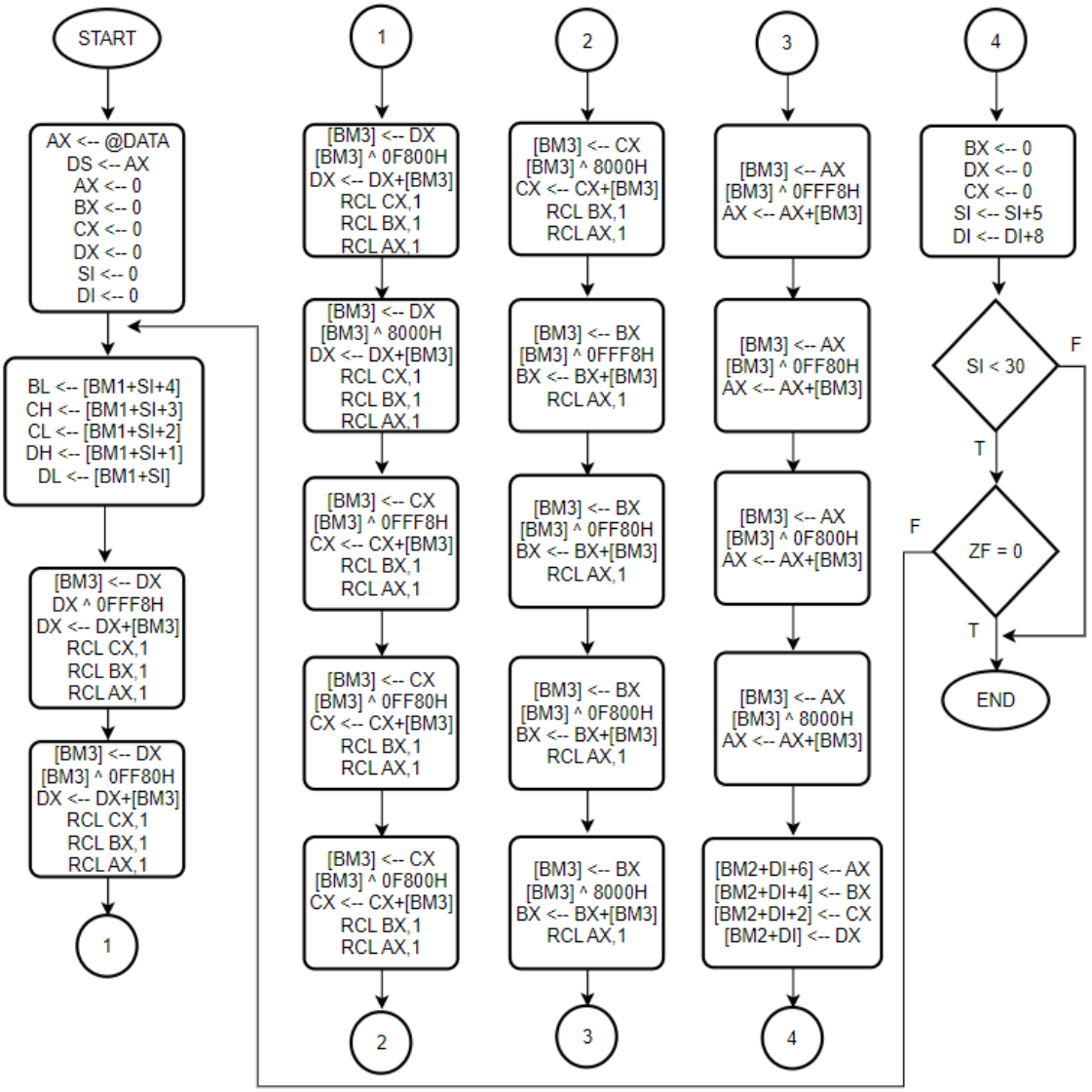
    MOV BX,0
    MOV DX,0
    MOV CX,0

    ADD SI,4
    ADD DI,6
    CMP SI,24
    JNZ LL1
    MOV AH,4CH
    INT 21H
    MAIN ENDP
END MAIN
```





7. Program 7


START
AX <-- @DATA
DS <-- AX
AX <-- 0
BX <-- 0
CX <-- 0
DX <-- 0
SI <-- 0
DI <-- 0
BL <-- [BM1+SI+4]
CH <-- [BM1+SI+3]
CL <-- [BM1+SI+2]
DH <-- [BM1+SI+1]
DL <-- [BM1+SI]
[BM3] <-- DX
DX ^ 0FFF8H
DX <-- DX+[BM3]
RCL CX,1
RCL BX,1
RCL AX,1
[BM3] <-- DX
[BM3] ^ 0FF80H
DX <-- DX+[BM3]
RCL CX,1
RCL BX,1
RCL AX,1
1
1
[BM3] <-- DX
[BM3] ^ 0F800H
DX <-- DX+[BM3]
RCL CX,1
RCL BX,1
RCL AX,1
[BM3] <-- DX
[BM3] ^ 8000H
DX <-- DX+[BM3]
RCL CX,1
RCL BX,1
RCL AX,1
[BM3] <-- CX
[BM3] ^ 0FFF8H
CX <-- CX+[BM3]
RCL BX,1
RCL AX,1
[BM3] <-- CX
[BM3] ^ 0FF80H
CX <-- CX+[BM3]
RCL BX,1
RCL AX,1
[BM3] <-- CX
[BM3] ^ 0F800H
CX <-- CX+[BM3]
RCL BX,1
RCL AX,1
2
2
[BM3] <-- CX
[BM3] ^ 8000H
CX <-- CX+[BM3]
RCL BX,1
RCL AX,1
[BM3] <-- BX
[BM3] ^ 0FFF8H
BX <-- BX+[BM3]
RCL AX,1
[BM3] <-- BX
[BM3] ^ 0FF80H
BX <-- BX+[BM3]
RCL AX,1
[BM3] <-- BX
[BM3] ^ 0F800H
BX <-- BX+[BM3]
RCL AX,1
[BM3] <-- BX
[BM3] ^ 8000H
BX <-- BX+[BM3]
RCL AX,1
3
3
[BM3] <-- AX
[BM3] ^ 0FFF8H
AX <-- AX+[BM3]
[BM3] <-- AX
[BM3] ^ 0FF80H
AX <-- AX+[BM3]
[BM3] <-- AX
[BM3] ^ 0F800H
AX <-- AX+[BM3]
[BM3] <-- AX
[BM3] ^ 8000H
AX <-- AX+[BM3]
[BM2+DI+6] <-- AX
[BM2+DI+4] <-- BX
[BM2+DI+2] <-- CX
[BM2+DI] <-- DX
4
4
BX <-- 0
DX <-- 0
CX <-- 0
SI <-- SI+5
DI <-- DI+8
SI < 30
F
T
ZF = 0
F
T
END

```
001 TITLE 6 DATA HEX 10D TO OCT ;IN BM1 PUT BM2
002 .MODEL SMALL
003 .STACK 100H
004 .DATA
005 BM1 DB 0ABH,0CDH,0BCH,0DEH,0CDH, 0EFH,0DEH,0FAH,11H,22H
006     DB 33H,44H,55H,66H,77H, 12H,0ABH,34H,0CDH,56H
007     DB 0EFH,78H,91H,0AAH,0BBH, 0CCH,0DDH,0EEH,0FFH,0BBH
008 BM2 DW 24 DUP(0)
009 BM3 DW 0H|
010
011 ;NAMA  : ELSADANIKA MAURA
012 ;NIM   : 211344013 - 2NK
013 .CODE
014 MAIN PROC
015       MOV AX,@DATA
016       MOV DS,AX
017       MOV AX,0
018       MOV BX,0
019       MOV CX,0
020       MOV DX,0
021       MOV SI,0
022       MOV DI,0
023
024
025 L1:   MOV BL,[BM1+SI+4]
026       MOV CH,[BM1+SI+3]
027       MOV CL,[BM1+SI+2]
028       MOV DH,[BM1+SI+1]
029       MOV DL,[BM1+SI]
030
031       MOV [BM3],DX
032       AND DX,0FFF8H
033       ADD DX,[BM3]
034       RCL CX,1
035       RCL BX,1
036       RCL AX,1
037
038       MOV [BM3],DX
039       AND [BM3],0FF80H
040       ADD DX,[BM3]
041       RCL CX,1
042       RCL BX,1
043       RCL AX,1
044
045       MOV [BM3],DX
046       AND [BM3],0F800H
047       ADD DX,[BM3]
048       RCL CX,1
049       RCL BX,1
050       RCL AX,1
051
052       MOV [BM3],DX
053       AND [BM3],8000H
054       ADD DX,[BM3]
055       RCL CX,1
056       RCL BX,1
057       RCL AX,1
058
```

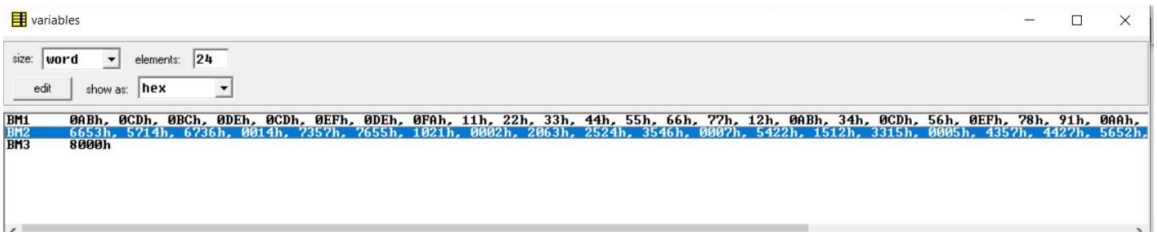
```
058
059
060       MOV [BM3],CX
061       AND [BM3],0FFF8H
062       ADD CX,[BM3]
063       RCL BX,1
064       RCL AX,1
065
066       MOV [BM3],CX
067       AND [BM3],0FF80H
068       ADD CX,[BM3]
069       RCL BX,1
070       RCL AX,1
071
072       MOV [BM3],CX
073       AND [BM3],0F800H
074       ADD CX,[BM3]
075       RCL BX,1
076       RCL AX,1
077
078       MOV [BM3],CX
079       AND [BM3],8000H
080       ADD CX,[BM3]
081       RCL BX,1
082       RCL AX,1
083
084
085       MOV [BM3],BX
086       AND [BM3],0FFF8H
087       ADD BX,[BM3]
088       RCL AX,1
089
090       MOV [BM3],BX
091       AND [BM3],0FF80H
092       ADD BX,[BM3]
093       RCL AX,1
094
095       MOV [BM3],BX
096       AND [BM3],0F800H
097       ADD BX,[BM3]
098       RCL AX,1
099
100       MOV [BM3],BX
101       AND [BM3],8000H
102       ADD BX,[BM3]
103       RCL AX,1
104
105
106       MOV [BM3],AX
107       AND [BM3],0FFF8H
108       ADD AX,[BM3]
109
110       MOV [BM3],AX
111       AND [BM3],0FF80H
112       ADD AX,[BM3]
113
114       MOV [BM3],AX
115       AND [BM3],0F800H
116       ADD AX,[BM3]
```

```
        ADD AX,[BM3]

        MOV [BM3],AX
        AND [BM3],8000H
        ADD AX,[BM3]

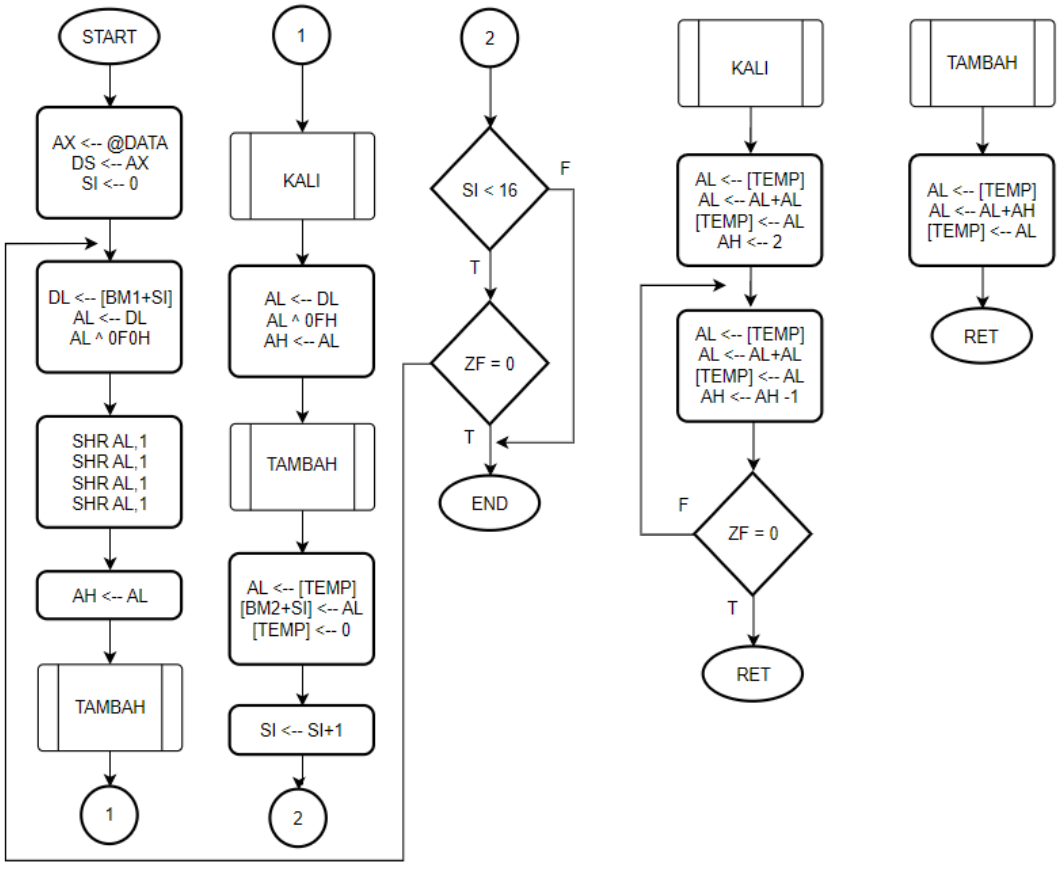

        MOV [BM2+DI+6],AX
        MOV [BM2+DI+4],BX
        MOV [BM2+DI+2],CX
        MOV [BM2+DI],DX
        MOV BX,0
        MOV CX,0
        MOV DX,0
        ADD SI,5
        ADD DI,8
        CMP SI,30
        JNZ L1

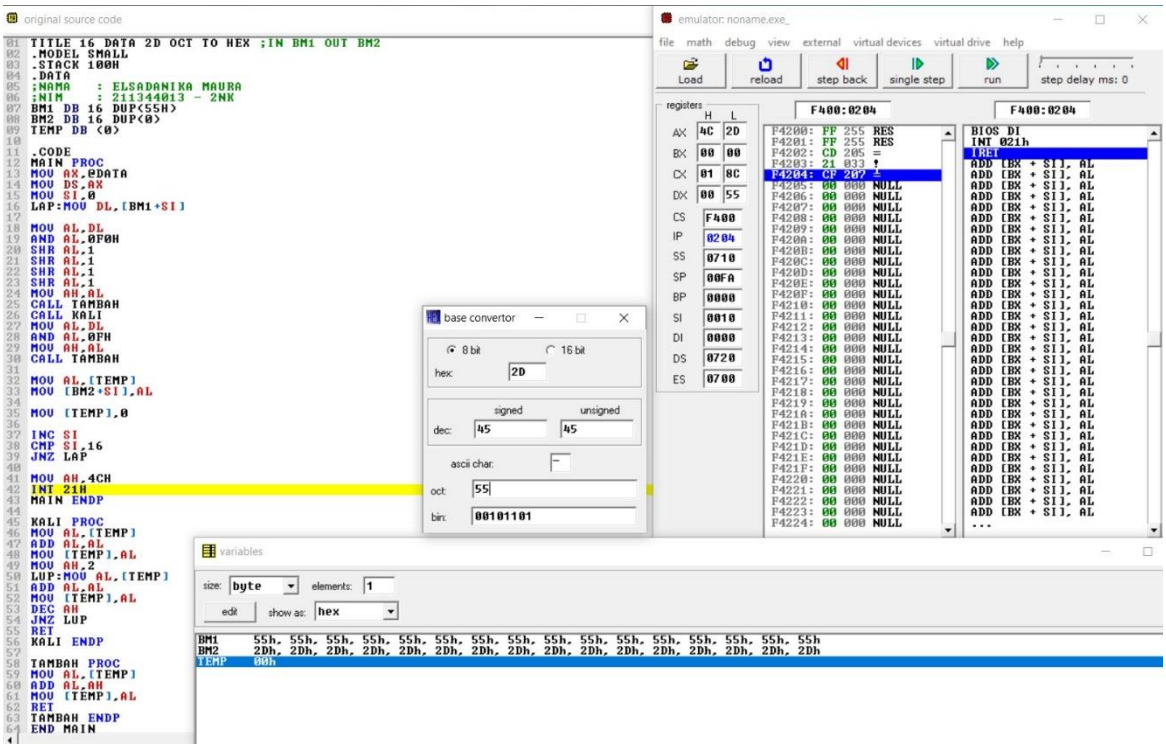
END MAIN
```



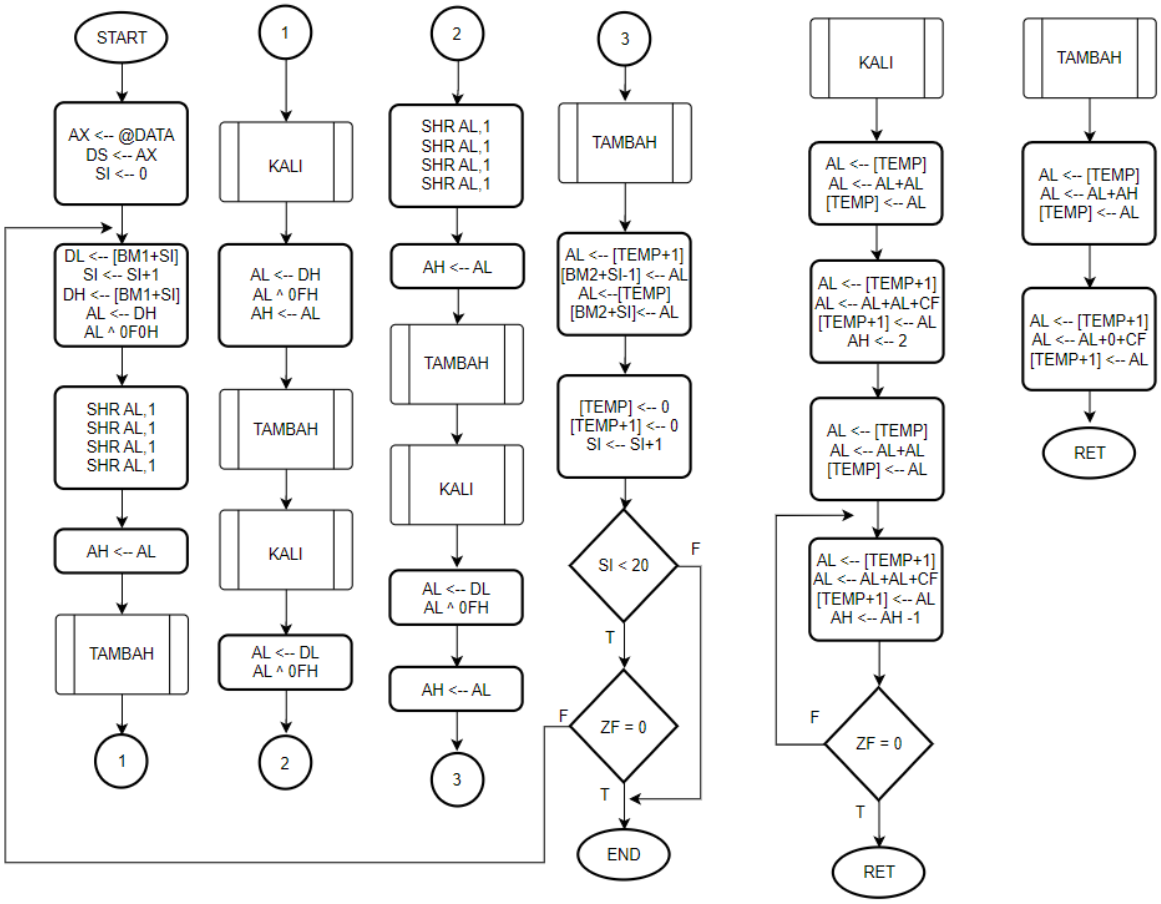


8. Program 8


START
AX <-- @DATA
DS <-- AX
SI <-- 0
DL <-- [BM1+SI]
AL <-- DL
AL ^ 0F0H
SHR AL,1
SHR AL,1
SHR AL,1
SHR AL,1
AH <-- AL
TAMBAH
1
1
KALI
AL <-- DL
AL ^ 0FH
AH <-- AL
TAMBAH
AL <-- [TEMP]
[BM2+SI] <-- AL
[TEMP] <-- 0
SI <-- SI+1
2
2
SI < 16
F
T
ZF = 0
T
END
KALI
AL <-- [TEMP]
AL <-- AL+AL
[TEMP] <-- AL
AH <-- 2
AL <-- [TEMP]
AL <-- AL+AL
[TEMP] <-- AL
AH <-- AH -1
F
ZF = 0
T
RET
TAMBAH
AL <-- [TEMP]
AL <-- AL+AH
[TEMP] <-- AL
RET


original source code
TITLE 16 DATA 2D OCT TO HEX ;IN BM1 OUT BM2
.MODEL SMALL
.STACK 100H
.DATA
;NAMA : ELSADANIKA MAURA
;NIM : 211344013 - 2NK
BM1 DB 16 DUP(55H)
BM2 DB 16 DUP(0)
TEMP DB (0)
.CODE
MAIN PROC
MOV AX,@DATA
MOV DS,AX
MOV SI,0
LAP:MOV DL,[BM1+SI]
MOV AL,DL
AND AL,0F0H
SHR AL,1
SHR AL,1
SHR AL,1
SHR AL,1
MOV AH,AL
CALL TAMBAH
CALL KALI
MOV AL,DL
AND AL,0FH
MOV AH,AL
CALL TAMBAH
MOV AL,[TEMP]
MOV [BM2+SI],AL
MOV [TEMP],0
INC SI
CMP SI,16
JNZ LAP
MOV AH,4CH
INT 21H
MAIN ENDP
KALI PROC
MOV AL,[TEMP]
ADD AL,AL
MOV [TEMP],AL
MOV AH,2
LUP:MOV AL,[TEMP]
ADD AL,AL
MOV [TEMP],AL
DEC AH
JNZ LUP
RET
KALI ENDP
TAMBAH PROC
MOV AL,[TEMP]
ADD AL,AH
MOV [TEMP],AL
RET
TAMBAH ENDP
END MAIN
emulator: noname.exe_
file math debug view external virtual devices virtual drive help
Load reload step back single step run step delay ms: 0
registers
base convertor
8 bit
16 bit
hex: 2D
signed
unsigned
dec: 45 45
ascii char:
oct: 55
bin: 00101101
variables
size: byte
elements: 1
edit
show as: hex
BM1 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h
BM2 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh, 2Dh
TEMP 00h

9. Program 9


START
AX <-- @DATA
DS <-- AX
SI <-- 0
DL <-- [BM1+SI]
SI <-- SI+1
DH <-- [BM1+SI]
AL <-- DH
AL ^ 0F0H
SHR AL,1
SHR AL,1
SHR AL,1
SHR AL,1
AH <-- AL
TAMBAH
1
1
KALI
AL <-- DH
AL ^ 0FH
AH <-- AL
TAMBAH
KALI
AL <-- DL
AL ^ 0FH
2
2
SHR AL,1
SHR AL,1
SHR AL,1
SHR AL,1
AH <-- AL
TAMBAH
KALI
AL <-- DL
AL ^ 0FH
AH <-- AL
3
3
TAMBAH
AL <-- [TEMP+1]
[BM2+SI-1] <-- AL
AL<--[TEMP]
[BM2+SI]<-- AL
[TEMP] <-- 0
[TEMP+1] <-- 0
SI <-- SI+1
SI < 20
F
T
ZF = 0
F
T
END
KALI
AL <-- [TEMP]
AL <-- AL+AL
[TEMP] <-- AL
AL <-- [TEMP+1]
AL <-- AL+AL+CF
[TEMP+1] <-- AL
AH <-- 2
AL <-- [TEMP]
AL <-- AL+AL
[TEMP] <-- AL
AL <-- [TEMP+1]
AL <-- AL+AL+CF
[TEMP+1] <-- AL
AH <-- AH -1
F
ZF = 0
T
RET
TAMBAH
AL <-- [TEMP]
AL <-- AL+AH
[TEMP] <-- AL
AL <-- [TEMP+1]
AL <-- AL+0+CF
[TEMP+1] <-- AL
RET


base convertor
8 bit
16 bit
hex:
B6D
signed
unsigned
dec:
2925
2925
ascii char:
oct:
5555
bin:
0000101101101101

```
TITLE 10 DATA 4D OCT TO HEX ;IN BM1 OUT BM2
.MODEL SMALL
.STACK 100H
.DATA
;NAMA    : ELSADANIKA MAURA SITORUS
;NIM     : 211344013 - 2NK
BM1 DB 20 DUP(55H)
BM2 DB 20 DUP(0)
TEMP DB 2 DUP(0)

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX
    MOV SI,0
LAP:MOV DL,[BM1+SI]
    INC SI
    MOV DH,[BM1+SI]

    MOV AL,DH
    AND AL,0F0H
    SHR AL,1
    SHR AL,1
    SHR AL,1
    SHR AL,1
    MOV AH,AL
    CALL TAMBAH
    CALL KALI
    MOV AL,DH
    AND AL,0FH
    MOV AH,AL
    CALL TAMBAH
    CALL KALI

    MOV AL,DL
    AND AL,0F0H
    SHR AL,1
    SHR AL,1
    SHR AL,1
    SHR AL,1
    MOV AH,AL
    CALL TAMBAH
    CALL KALI
    MOV AL,DL
    AND AL,0FH
    MOV AH,AL
    CALL TAMBAH

    MOV AL,[TEMP+1]
    MOV [BM2+SI-1],AL
    MOV AL,[TEMP]
    MOV [BM2+SI],AL

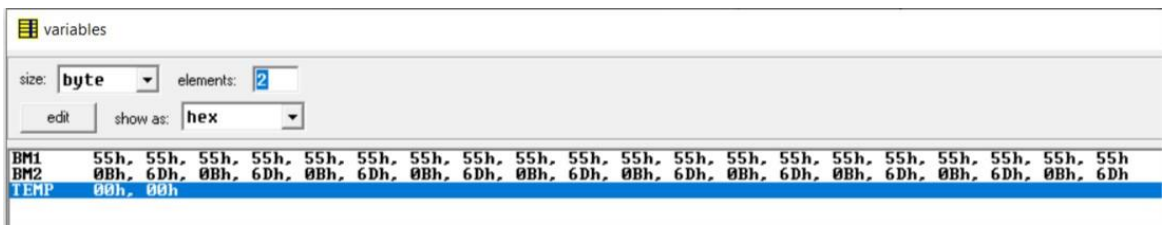
    MOV [TEMP],0
    MOV [TEMP+1],0

    INC SI
    CMP SI,20
    JNZ LAP

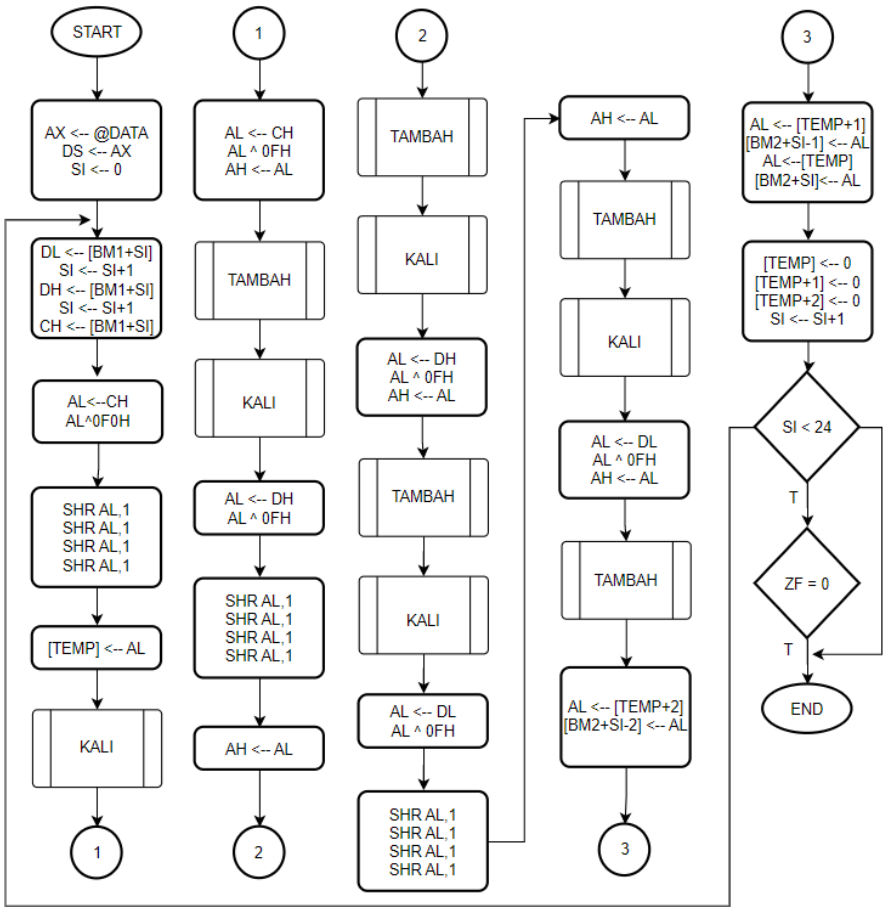
    MOV AH,4CH
    INT 21H
    MAIN ENDP

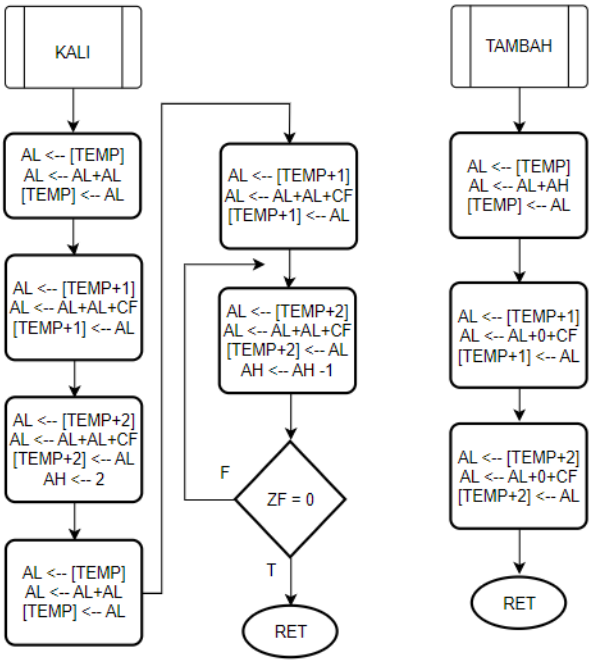
KALI PROC
    MOV AL,[TEMP]
    ADD AL,AL
    MOV [TEMP],AL
    MOV AL,[TEMP+1]
    ADC AL,AL
    MOV [TEMP+1],AL
    MOV AH,2
LUP:MOV AL,[TEMP]
    ADD AL,AL
    MOV [TEMP],AL
    MOV AL,[TEMP+1]
    ADC AL,AL
    MOV [TEMP+1],AL
    DEC AH
    JNZ LUP
    RET
    KALI ENDP

TAMBAH PROC
    MOV AL,[TEMP]
    ADD AL,AH
    MOV [TEMP],AL
    MOV AL,[TEMP+1]
    ADC AL,0
    MOV [TEMP+1],AL
    RET
    TAMBAH ENDP
END MAIN
```



10. Program 10

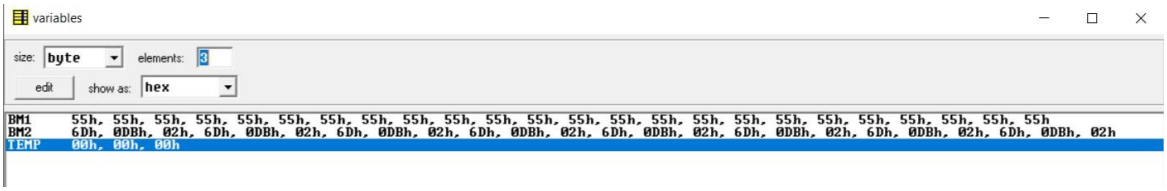


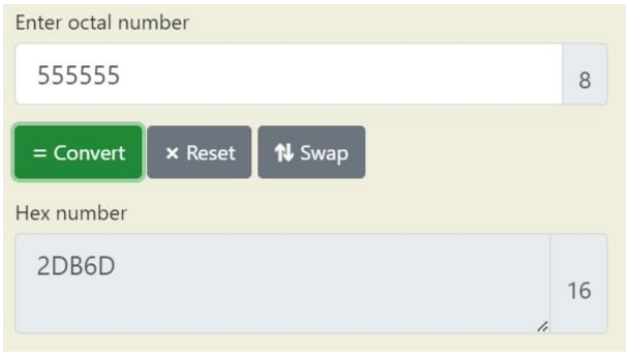


```
001 TITLE 8 DATA 6D OCT TO HEX
002 .MODEL SMALL
003 .STACK 100H
004 .DATA
005 BM1 DB 24 DUP(55)
006 BM2 DB 24 DUP(0)
007 TEMP DB 3 DUP(0)
008
009 ;NAMA    : ELSADANIKA MAURA
010 ;NIM     : 211344013 - 2NK
011
012 .CODE
013 MAIN PROC
014     MOV AX,@DATA
015     MOV DS,AX
016     MOV SI,0
017 LAP:MOV DL,[BM1+SI]
018     INC SI
019     MOV DH,[BM1+SI]
020     INC SI
021     MOV CH,[BM1+SI]
022
023     MOV AL,CH
024     AND AL,0F0H
025     SHR AL,1
026     SHR AL,1
027     SHR AL,1
028     SHR AL,1
029     MOV [TEMP],AL
030     CALL KALI
031     MOV AL,CH
032     AND AL,0FH
033     MOV AH,AL
034     CALL TAMBAH
035     CALL KALI
036
037     MOV AL,DH
038     AND AL,0F0H
039     SHR AL,1
040     SHR AL,1
041     SHR AL,1
042     SHR AL,1
043     MOV AH,AL
044     CALL TAMBAH
045     CALL KALI
046     MOV AL,DH
047     AND AL,0FH
048     MOV AH,AL
049     CALL TAMBAH
050     CALL KALI
051
052     MOV AL,DL
053     AND AL,0F0H
054     SHR AL,1
055     SHR AL,1
056     SHR AL,1
057     SHR AL,1
058     MOV AH,AL
```

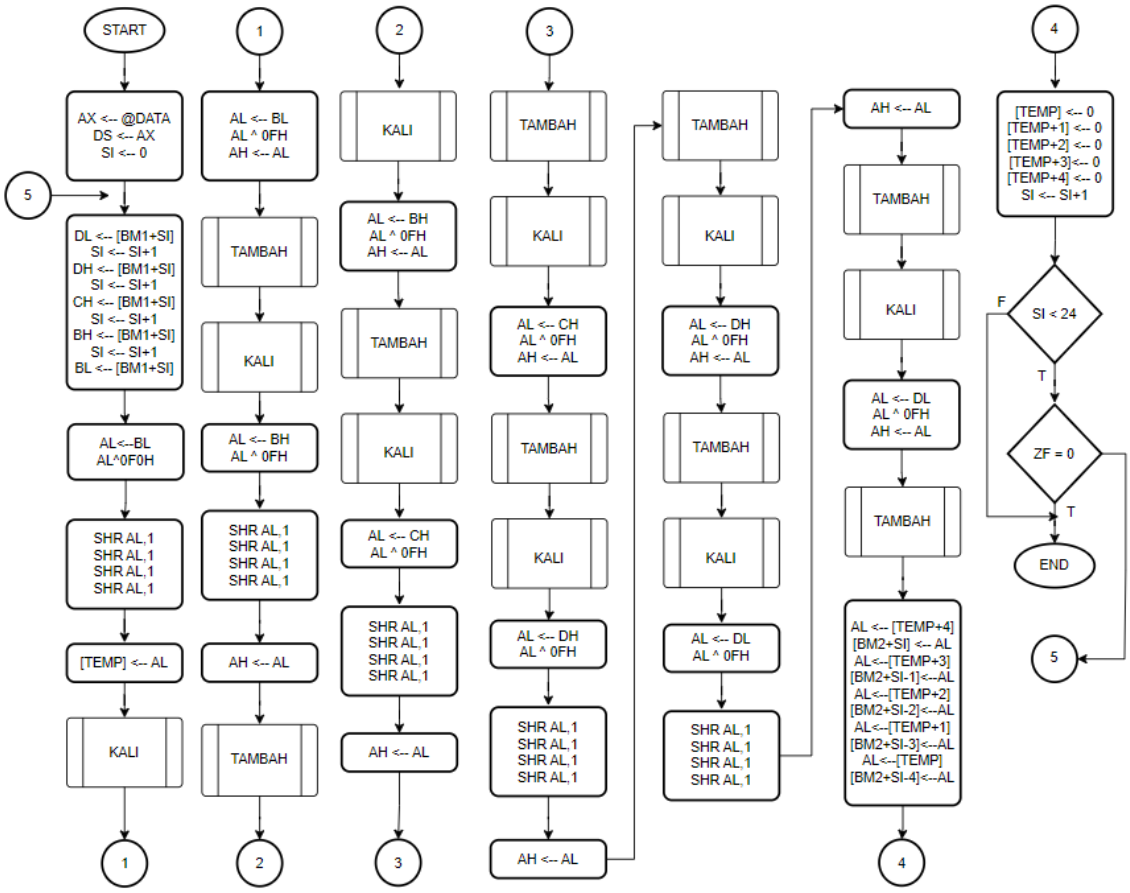
```
058     MOV AH,AL
059     CALL TAMBAH
060     CALL KALI
061     MOV AL,DL
062     AND AL,0FH
063     MOV AH,AL
064     CALL TAMBAH
065
066     MOV AL,[TEMP+2]
067     MOV [BM2+SI],AL
068     MOV AL,[TEMP+1]
069     MOV [BM2+SI-1],AL
070     MOV AL,[TEMP]
071     MOV [BM2+SI-2],AL
072
073     MOV [TEMP],0
074     MOV [TEMP+1],0
075     MOV [TEMP+2],0
076
077     INC SI
078     CMP SI,24
079     JNZ LAP
080
081     MOV AH,4CH
082     INT 21H
083     MAIN ENDP
084
085 KALI PROC
086     MOV AL,[TEMP]
087     ADD AL,AL
088     MOV [TEMP],AL
089     MOV AL,[TEMP+1]
090     ADC AL,AL
091     MOV [TEMP+1],AL
092     MOV AL,[TEMP+2]
093     ADC AL,AL
094     MOV [TEMP+2],AL
095     MOV AH,2
096 LUP:MOV AL,[TEMP]
097     ADD AL,AL
098     MOV [TEMP],AL
099     MOV AL,[TEMP+1]
100     ADC AL,AL
101     MOV [TEMP+1],AL
102     MOV AL,[TEMP+2]
103     ADC AL,AL
104     MOV [TEMP+2],AL
105     DEC AH
106     JNZ LUP
107     RET
108     KALI ENDP
109
```

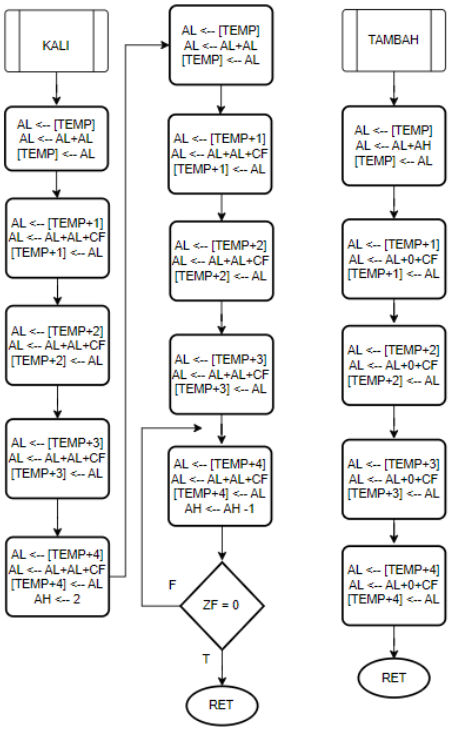
```
110 TAMBAH PROC
111     MOV AL,[TEMP]
112     ADD AL,AH
113     MOV [TEMP],AL
114     MOV AL,[TEMP+1]
115     ADC AL,0
116     MOV [TEMP+1],AL
117     MOV AL,[TEMP+2]
118     ADC AL,0
119     MOV [TEMP+2],AL
120     RET
121     TAMBAH ENDP
122 END MAIN
```





11. Program 11





```
001 TITLE OCT TO HEX 6DATA 10D
002 .MODEL SMALL
003 .STACK 100H
004 .DATA
005 ;NAMA  : ELSADANIKA MAURA
006 ;NIM   : 211344013 - 2NK
007 BM1 DB 30 DUP(55H)
008 BM2 DB 30 DUP(0)
009 TEMP DB 5 DUP(0)
010
011 .CODE
012 MAIN PROC
013     MOV AX,@DATA
014     MOV DS,AX
015     MOV SI,0
016 LAP:MOV DL,[BM1+SI]
017     INC SI
018     MOV DH,[BM1+SI]
019     INC SI
020     MOV CH,[BM1+SI]
021     INC SI
022     MOV BH,[BM1+SI]
023     INC SI
024     MOV BL,[BM1+SI]
025
026     MOV AL,BL
027     AND AL,0F0H
028     SHR AL,1
029     SHR AL,1
030     SHR AL,1
031     SHR AL,1
032     MOV [TEMP],AL
033     CALL KALI
034     MOV AL,BL
035     AND AL,0FH
036     MOV AH,AL
037     CALL TAMBAH
038     CALL KALI
039
040     MOV AL,BH
041     AND AL,0F0H
042     SHR AL,1
043     SHR AL,1
044     SHR AL,1
045     SHR AL,1
046     MOV AH,AL
047     CALL TAMBAH
048     CALL KALI
049     MOV AL,BH
050     AND AL,0FH
051     MOV AH,AL
052     CALL TAMBAH
053     CALL KALI
054
055     MOV AL,CH
056     AND AL,0F0H
057     SHR AL,1
058     SHR AL,1
059     SHR AL,1
060     SHR AL,1
061     MOV AH,AL
062     CALL TAMBAH
063     CALL KALI
064     MOV AL,CH
065     AND AL,0FH
066     MOV AH,AL
067     CALL TAMBAH
068     CALL KALI
069
070     MOV AL,DH
071     AND AL,0F0H
072     SHR AL,1
073     SHR AL,1
074     SHR AL,1
075     SHR AL,1
076     MOV AH,AL
077     CALL TAMBAH
078     CALL KALI
079     MOV AL,DH
080     AND AL,0FH
081     MOV AH,AL
082     CALL TAMBAH
083     CALL KALI
084
085     MOV AL,DL
086     AND AL,0F0H
087     SHR AL,1
088     SHR AL,1
089     SHR AL,1
090     SHR AL,1
091     MOV AH,AL
092     CALL TAMBAH
093     CALL KALI
094     MOV AL,DL
095     AND AL,0FH
096     MOV AH,AL
097     CALL TAMBAH
098
099     MOV AL,[TEMP+4]
100     MOV [BM2+SI],AL
101     MOV AL,[TEMP+3]
102     MOV [BM2+SI-1],AL
103     MOV AL,[TEMP+2]
104     MOV [BM2+SI-2],AL
105     MOV AL,[TEMP+1]
106     MOV [BM2+SI-3],AL
107     MOV AL,[TEMP]
108     MOV [BM2+SI-4],AL
109
110     MOV [TEMP],0
111     MOV [TEMP+1],0
112     MOV [TEMP+2],0
113     MOV [TEMP+3],0
114     MOV [TEMP+4],0
```

```
    INC SI
    CMP SI,30
    JNZ LAP

    MOV AH,4CH
    INT 21H
    MAIN ENDP

KALI PROC
    MOV AL,[TEMP]
    ADD AL,AL
    MOV [TEMP],AL
    MOV AL,[TEMP+1]
    ADC AL,AL
    MOV [TEMP+1],AL
    MOV AL,[TEMP+2]
    ADC AL,AL
    MOV [TEMP+2],AL
    MOV AL,[TEMP+3]
    ADC AL,AL
    MOV [TEMP+3],AL
    MOV AL,[TEMP+4]
    ADC AL,AL
    MOV [TEMP+4],AL
    MOV AH,2
LUP:MOV AL,[TEMP]
    ADD AL,AL
    MOV [TEMP],AL
    MOV AL,[TEMP+1]
    ADC AL,AL
    MOV [TEMP+1],AL
    MOV AL,[TEMP+2]
    ADC AL,AL
    MOV [TEMP+2],AL
    MOV AL,[TEMP+3]
    ADC AL,AL
    MOV [TEMP+3],AL
    MOV AL,[TEMP+4]
    ADC AL,AL
    MOV [TEMP+4],AL
    DEC AH
    JNZ LUP
    RET
    KALI ENDP

TAMBAH PROC
    MOV AL,[TEMP]
    ADD AL,AH
    MOV [TEMP],AL
    MOV AL,[TEMP+1]
    ADC AL,0
    MOV [TEMP+1],AL
    MOV AL,[TEMP+2]
    ADC AL,0
    MOV [TEMP+2],AL
    MOV AL,[TEMP+3]
    ADC AL,0
    MOV [TEMP+3],AL
    MOV AL,[TEMP+4]
    ADC AL,0
    MOV [TEMP+4],AL
    RET
    TAMBAH ENDP
END MAIN
```

size: byte elements: 5

edit show as: hex

```
BM1     55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55h, 55
BM2     6Dh, 0DBh, 0B6h, 2Dh, 00h, 6Dh, 0DBh, 0B6h, 2Dh, 00h, 6Dh, 0DBh, 0B6h, 2Dh, 00h, 6Dh, 0DBh, 0B6h, 2Dh, 00h, 6Dh, 0DBh, 0B6h, 2Dh, 00
TEMP    00h, 00h, 00h, 00h, 00h
```

Enter octal number

5555555555 8

= Convert × Reset ⇅ Swap

Hex number

2DB6DB6D 16

F. Analisis Data

Praktikum konversi data-data bilangan heksadesimal ke oktal dan oktal ke heksadesimal pada intel 8086 menggunakan simulator EMU8086 yang telah dilakukan menghasilkan data yang sesuai antara perhitungan teori/manual/calculator dengan perhitungan berdasarkan program yang telah dibuat. Konversi data heksadesimal ke oktal menggunakan konsep penyisipan 0 untuk setiap 3 digit data heksadesimal dalam biner. Cara kerja konsep ini adalah seperti contoh berikut. Bilangan heksadesimal $3F_{(16)}$ = 00111111. Bilangan biner dari 3F disusun per 3 bit menjadi 00 111 111 sehingga hasilnya adalah 077 dalam oktal. Penyisipan 0 pada tiap 3 bit data bekerja dalam bentuk binernya menjadi 000 01110111 sehingga nilai oktalnya tetap 77, tidak berubah.

Pada program pertama, data dua digit akan dikonversi dari heksadesimal ke oktal. Data yang diinputkan melalui register CH akan menghasilkan output di register DH. Langkah pertama adalah memindahkan data ke register lain untuk mengamankan data sehingga data asli/murni tidak rusak oleh proses konversi. Data yang telah dipindahkan ke register lain, kemudian di AND kan dengan 0F8H untuk membuat 3 digit terakhir dari data menjadi 0. Setelah proses tersebut, dilakukan penumlahan dengan data murni CH lalu carry disimpan di DH. Hasil dari proses yang telah dilakukan dipindahkan ke register BX lalu kembali di AND kan dengan 0FF80H untuk kembali membuat 3 digit terakhir menjadi 0 dan dijumlahkan lagi dengan data pada register pengaman. Dari langkah tersebut, maka akan didapatkan bilangan biner yang telah tersisipkan angka 0 sehingga terkonversi menjadi bilangan oktal.

Pada program kedua, memiliki konsep yang sama dengan program pertama, akan tetapi data memiliki 4 digit sehingga hanya perlu menambahlan faktor pengali dengan 0FFF8H. Setelah itu dijumlahkan kembali dengan data murni yang diinputkan melalui register CX. Digunakan perintah RCL CL,1 untuk menggeser CL ke kiri sehingga didapat 1 space kosong untuk menyisipkan 0. Untuk menyisipkan angka 0 maka dilakukan proses AND dengan 0FF80H, 0F800H, dan 8000H sehingga akan dilakukan penyisipan 0 untuk setiap 3 digit dari data. Dari proses ini, didapatkan hasil data terkonversi pada register DX sebanyak 4 digit, akan tetapi untuk 4 digit heksadesimal ke oktal masih diperlukan 1 register tambahan yaitu CL. Untuk register CL kembali dilakukan pengali dengan 0F8H untuk menyisipkan 0 pada 3 digit datanya

sehingga dengan demikian didapatkan hasil bilangan oktal yang dikonversi dari heksadesimal.

Untuk program 3 hingga 7, masih digunakan konsep yang sama dengan program 1 dan 2. Perbedaan yang terdapat adalah penambahan proses AND di setiap register baru dikarenakan data yang dimiliki adalah lebih dari 4 digit, sehingga akan dibutuhkan lebih dari 2 register 16 bit serta jumlah penggunaan perintah RCL yang menyesuaikan dengan digit yang dimiliki dari tiap data. Data disimpan pada data area BM1 dan ditampilkan hasil konversinya pada data area BM2. Selain itu juga digunakan perintah CMP SI untuk pengulangan yang terjadi pada loop konversi. Untuk data dengan jumlah banyak, digunakan register DI sebagai indeks tambahan pada proses loopingnya.

Pada program konversi data dari bilangan oktal ke heksadesimal, digunakan konsep penggandaan. Data oktal memiliki basis 8 sehingga hanya perlu penggandaan sebanyak 3 kali untuk mendapat faktor pengali 8. Konsep penggandaan untuk mendapat pengali 8 adalah sebagai berikut:

(A*8) = A * (2+2+2)

2 = penggandaan 1 kali

Sehingga data perlu digandakan sebanyak 3 kali untuk mendapat hasil yang sama dengan pengali 8.

Untuk program dengan data-data yang sudah diketahui dan diberikan di data area serta diinginkan output di data area, maka perlu pendeklarasian data area. Data area dideklarasikan sebagai BM1 DB untuk semua data yang dibentuk dalam 8 bit atau BM1 DW untuk semua data yang dibentuk dalam 16 bit. Selain pendeklarasianBM1, diperlukan juga BM2 DB atau BM2 DW untuk menampung hasil konversi heksadesimal dari oktal. Memory area TEMP digunakan dalam program ini. Program 8 hingga 11 memiliki konsep yang sama melalui proses penggandaan. Data dari data area BM1 dipindahkan ke register 8 bit untuk data 8 bit atau register 16 bit untuk data 16 bit dengan increment SI setiap perpindahannya. Sedangkan setiap hasil proses sementara disimpan di TEMP sebelum dipindahkan ke BM2 pada akhir prosedur MAIN.

Prosedur KALI pada program 8 menggunakan memory area TEMP. Data pada TEMP dipindahkan ke register AL, lalu dijumlahkan kembali dengan register AL. Setelah dijumlahkan, data dikembalikan ke TEMP. Prosedur perkalian menggunakan lup, dengan decrement sebanyak 2 kali yang disimpan pada register AH. Loop dimulai

dengan memindakan data pada TEMP ke AL, lalu dijumlahkan kembali dengan AL dan data hasil penjumlahan dikembalikan ke TEMP lalu akan kembali looping hingga sebanyak 2 kali.

Digunakan juga prosedur tambah yang diisi dengan penjumlahan data AH ke register AL yang diambil dari TEMP dan dikembalikan ke TEMP. Jumlah TEMP pada prosedur KALI dan TAMBAH akan terus bertambah seiring dengan penambahan jumlah digit pada data. TEMP digunakan untuk melakukan setiap proses dan disimpan sementara di TEMP. Dengan demikian program konversi telah selesai dilakukan dan didapatkan hasil yang sesuai antara program dengan teori dan/atau kalkulator konversi.

G. Kesimpulan

Melalui praktikum yang telah dilakukan, diketahui bahwa didalam program konversi data dari oktal ke heksadesimal memerlukan faktor pengali 8. Untuk mendapatkan faktor pengali 8 ini, digunakan konsep penggandaan data yang dilakukan sebanyak 3 kali untuk mendapat faktor 8. Sehingga pada program konversi oktal ke heksadesimal program kelima yaitu data dua digit memiliki konsep yang sama dengan decimal ke heksadesimal program pertama, hanya perlu menghilangkan faktor pengali 1 yaitu pengganda 2 dan perintah-perintah yang terkait dengan pengganda 2.

Selain itu, untuk mempermudah proses konversi, dapat digunakan memory area tambahan. Dalam hal ini, digunakan memory area TEMP untuk proses operasi sementara. TEMP digunakan untuk melakukan setiap proses dan disimpan sementara di TEMP. Hal penting lainnya adalah, diperlukan pemisahan satuan dan puluhan pada decimal dan satuan dengan delapanan pada oktal sebelum dikenakan proses penggandaan. Hal ini dilakukan supaya data tidak menjadi rusak dan menghasilkan output yang tidak tepat.

Konversi data heksadesimal ke oktal menggunakan konsep penyisipan 0 untuk setiap 3 digit data heksadesimal dalam biner. Penyisipan 0 pada tiap 3 bit data bekerja dalam bentuk binernya sehingga nilai oktalnya tetap dan tidak berubah. Digunakan perintah RCL untuk menggeser register 8 bit ke kiri sehingga didapat 1 space kosong untuk menyisipkan 0. Untuk menyisipkan angka 0 maka dilakukan proses AND dengan 0FF80H, 0F800H, dan 8000H sehingga akan dilakukan penyisipan 0 untuk setiap 3 digit

dari data. Dengan demikian program-program konversi telah selesai dilakukan dan didapatkan hasil yang sesuai antara program dengan teori dan/atau kalkulator konversi.

H. Daftar Pustaka

[1] Kho, D. Pengertian Mikroprosesor (Microprocessor) dan Cara Kerja Mikroprossesor. [teknikelektronika.com]. Available: https://teknikelektronika.com/pengertian-mikroprosesor-microprocessor-cara-kerja-mikroprosesor-adalah/

[2] Ardiyanto88. 2013. Perintah Dasar Bahasa Assembly Menggunakan Emulator 8086. [ardiyanto88.wordpress.com]. Available: https://ardiyanto88.wordpress.com/2013/10/25/perintah-dasar-bahasa-assembly-menggunakan-emulator-8086/